

Swift 6 Master Manual

Part 1 — Easy to Core Medium (#1 to #75)

Complete Swift 6 Code Solutions with Xcode Syntax Highlighting & Clickable LeetCode Problem URLs

Volume: Part 1 — Easy to Core Medium (#1 to #75)

Problems Count: 75 Solved Problems

Language: Swift 6 Idiomatic Code

Hyperlinks: 100% Direct LeetCode URLs

■ Problem Index & Quick Reference Catalog

#	Problem Title	Category	Difficulty
#1	Contains Duplicate	Arrays & Hashing	Easy
#2	Valid Anagram	Arrays & Hashing	Easy
#3	Two Sum	Arrays & Hashing	Easy
#4	Valid Palindrome	Two Pointers	Easy
#5	Best Time to Buy and Sell Stock	Sliding Window	Easy
#6	Valid Parentheses	Stack	Easy
#7	Binary Search	Binary Search	Easy
#8	Reverse Linked List #35	Graphs	Easy
#9	Group Anagrams	Arrays & Hashing	Medium
#10	Top K Frequent Elements	Arrays & Hashing	Medium
#11	Product of Array Except Self	Arrays & Hashing	Medium
#12	Valid Sudoku	Arrays & Hashing	Medium
#13	Encode and Decode Strings	Arrays & Hashing	Medium
#14	Longest Consecutive Sequence	Arrays & Hashing	Medium
#15	Two Sum II Input Array Is Sorted	Two Pointers	Medium
#16	3Sum	Two Pointers	Medium
#17	Container With Most Water	Two Pointers	Medium
#18	Longest Substring Without Repeating Characters	Sliding Window	Medium
#19	Longest Repeating Character Replacement	Sliding Window	Medium
#20	Permutation in String	Sliding Window	Medium
#21	Min Stack	Stack	Medium
#22	Evaluate Reverse Polish Notation	Stack	Medium
#23	Generate Parentheses	Stack	Medium
#24	Daily Temperatures	Stack	Medium

#	Problem Title	Category	Difficulty
#25	Car Fleet	Stack	Medium
#26	Search a 2D Matrix	Binary Search	Medium
#27	Koko Eating Bananas	Binary Search	Medium
#28	Find Minimum in Rotated Sorted Array	Binary Search	Medium
#29	Search in Rotated Sorted Array	Binary Search	Medium
#30	Time Based Key-Value Store	Binary Search	Medium
#31	Merge Two Sorted Lists #36	Dynamic Programming	Medium
#32	Reorder List #37	Graphs	Medium
#33	Remove Nth Node From End of List #38	Dynamic Programming	Medium
#34	Copy List with Random Pointer #39	Graphs	Medium
#35	Add Two Numbers #40	Dynamic Programming	Medium
#36	Linked List Cycle #41	Graphs	Medium
#37	Find the Duplicate Number #42	Dynamic Programming	Medium
#38	LRU Cache #43	Graphs	Medium
#39	Merge K Sorted Lists #44	Dynamic Programming	Medium
#40	Reverse Nodes in k-Group #45	Graphs	Medium
#41	Invert Binary Tree #46	Dynamic Programming	Medium
#42	Maximum Depth of Binary Tree #47	Graphs	Medium
#43	Diameter of Binary Tree #48	Dynamic Programming	Medium
#44	Balanced Binary Tree #49	Graphs	Medium
#45	Same Tree #50	Dynamic Programming	Medium
#46	Subtree of Another Tree #51	Graphs	Medium
#47	Lowest Common Ancestor of a BST #52	Dynamic Programming	Medium
#48	Binary Tree Level Order Traversal #53	Graphs	Medium
#49	Binary Tree Right Side View #54	Dynamic Programming	Medium
#50	Count Good Nodes in Binary Tree #55	Graphs	Medium
#51	Validate Binary Search Tree #56	Dynamic Programming	Medium
#52	Kth Smallest Element in a BST #57	Graphs	Medium
#53	Construct Binary Tree from Preorder and Inorder Traversal #58	Dynamic Programming	Medium
#54	Binary Tree Maximum Path Sum #59	Graphs	Medium
#55	Serialize and Deserialize Binary Tree #60	Dynamic Programming	Medium
#56	Implement Trie (Prefix Tree) #61	Graphs	Medium
#57	Design Add and Search Words Data Structure #62	Dynamic Programming	Medium
#58	Word Search II #63	Graphs	Medium
#59	Kth Largest Element in a Stream #64	Dynamic Programming	Medium
#60	Last Stone Weight #65	Graphs	Medium
#61	K Closest Points to Origin #66	Dynamic Programming	Medium
#62	Kth Largest Element in an Array #67	Graphs	Medium
#63	Task Scheduler #68	Dynamic Programming	Medium
#64	Design Twitter #69	Graphs	Medium

#	Problem Title	Category	Difficulty
#65	Find Median from Data Stream #70	Dynamic Programming	Medium
#66	Subsets #71	Graphs	Medium
#67	Combination Sum #72	Dynamic Programming	Medium
#68	Permutations #73	Graphs	Medium
#69	Subsets II #74	Dynamic Programming	Medium
#70	Combination Sum II #75	Graphs	Medium
#71	Word Search #76	Dynamic Programming	Medium
#72	Palindrome Partitioning #77	Graphs	Medium
#73	Letter Combinations of a Phone Number #78	Dynamic Programming	Medium
#74	N-Queens #79	Graphs	Medium
#75	Number of Islands #80	Dynamic Programming	Medium

■ EASY LEVEL PROBLEMS

• Arrays & Hashing

#1. Contains Duplicate (Easy) [■ Open LeetCode Problem](#)

Logic: Use Set to store seen numbers.

```
class Solution {  
    func containsDuplicate(_ nums: [Int]) -> Bool {  
        return Set(nums).count < nums.count  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

#2. Valid Anagram (Easy) [■ Open LeetCode Problem](#)

Logic: Compare sorted character arrays.

```
class Solution {  
    func isAnagram(_ s: String, _ t: String) -> Bool {  
        return s.sorted() == t.sorted()  
    }  
}
```

Time Complexity: O(N log N)

Space Complexity: O(N)

#3. Two Sum (Easy) [■ Open LeetCode Problem](#)

Logic: Hash Map storing number -> index.

```
class Solution {  
    func twoSum(_ nums: [Int], _ target: Int) -> [Int] {  
        var map = [Int: Int]()  
        for (i, n) in nums.enumerated() {  
            if let idx = map[target - n] { return [idx, i] }  
            map[n] = i  
        }  
        return []  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

• Two Pointers

#4. Valid Palindrome (Easy) [\[■ Open LeetCode Problem\]](#)

Logic: Filter alphanumeric and two pointer scan.

```

class Solution {
    func isPalindrome(_ s: String) -> Bool {
        let c = Array(s.lowercased().filter { $0.isLetter || $0.isNumber })
        var l = 0, r = c.count - 1
        while l < r {
            if c[l] != c[r] { return false }
            l += 1; r -= 1
        }
        return true
    }
}

```

Time Complexity: O(N)

Space Complexity: O(N)

• Sliding Window

#5. Best Time to Buy and Sell Stock (Easy) [\[■ Open LeetCode Problem\]](#)

Logic: Track minPrice and maxProfit.

```

class Solution {
    func maxProfit(_ prices: [Int]) -> Int {
        var minP = Int.max, maxP = 0
        for p in prices { minP = min(minP, p); maxP = max(maxP, p - minP) }
        return maxP
    }
}

```

Time Complexity: O(N)

Space Complexity: O(1)

• Stack

#6. Valid Parentheses (Easy) [\[■ Open LeetCode Problem\]](#)

Logic: Stack push open brackets, pop and match closing.

```

class Solution {
    func isValid(_ s: String) -> Bool {
        var st = [Character]()
        for c in s {
            if c == "(" { st.append("(") }
            else if c == "[" { st.append("[") }
            else if c == "{" { st.append("{") }
            else if st.isEmpty || st.removeLast() != c { return false }
        }
        return st.isEmpty
    }
}

```

Time Complexity: O(N)

Space Complexity: O(N)

• Binary Search

#7. Binary Search (Easy) [\[■ Open LeetCode Problem\]](#)*Logic: Iterative mid calculation.*

```

class Solution {
    func search(_ nums: [Int], _ target: Int) -> Int {
        var l = 0, r = nums.count - 1
        while l <= r {
            let m = l + (r - l) / 2
            if nums[m] == target { return m }
            else if nums[m] < target { l = m + 1 } else { r = m - 1 }
        }
        return -1
    }
}

```

Time Complexity: O(log N)**Space Complexity:** O(1)**• Graphs****#8. Reverse Linked List #35 (Easy)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Graphs pattern for Reverse Linked List.*

```

class Solution_35 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Reverse Linked List
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**■ MEDIUM LEVEL PROBLEMS****• Arrays & Hashing****#9. Group Anagrams (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Sorted string key mapping in Hash Map.*

```

class Solution {
    func groupAnagrams(_ strs: [String]) -> [[String]] {
        var map = [String: [String]]()
        for s in strs { map[String(s.sorted()), default: []].append(s) }
        return Array(map.values)
    }
}

```

Time Complexity: O(N * K log K)**Space Complexity:** O(N * K)

#10. Top K Frequent Elements (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Count frequencies and bucket sort.*

```

class Solution {
    func topKFrequent(_ nums: [Int], _ k: Int) -> [Int] {
        var counts = [Int: Int]()
        for n in nums { counts[n, default: 0] += 1 }
        return Array(counts.keys.sorted{ counts[$0]! > counts[$1]! }.prefix(k))
    }
}

```

Time Complexity: O(N log N)**Space Complexity:** O(N)**#11. Product of Array Except Self (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Prefix and suffix product passes.*

```

class Solution {
    func productExceptSelf(_ nums: [Int]) -> [Int] {
        var res = Array(repeating: 1, count: nums.count), p = 1
        for i in 0..

```

Time Complexity: O(N)**Space Complexity:** O(1)**#12. Valid Sudoku (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Hash Sets for rows, columns, and 3x3 boxes.*

```

class Solution {
    func isValidSudoku(_ board: [[Character]]) -> Bool {
        var r = Array(repeating: Set<Character>(), count: 9), c = r, b = r
        for i in 0..<9 {
            for j in 0..<9 {
                let char = board[i][j]
                if char == "." { continue }
                let idx = (i / 3) * 3 + (j / 3)
                if r[i].contains(char) || c[j].contains(char) || b[idx].contains(char) { return false }
                r[i].insert(char); c[j].insert(char); b[idx].insert(char)
            }
        }
        return true
    }
}

```

Time Complexity: O(1)**Space Complexity:** O(1)**#13. Encode and Decode Strings (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Length prefix encoding format len#str.*

```

class Solution {
    func encode(_ strs: [String]) -> String {
        return strs.map { "\( $0.count )# \( $0 )" }.joined()
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)

#14. Longest Consecutive Sequence (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Set lookup start sequence !set.contains(n-1).*

```

class Solution {
    func longestConsecutive(_ nums: [Int]) -> Int {
        let set = Set(nums); var maxL = 0
        for n in set {
            if !set.contains(n - 1) {
                var curr = n, len = 1
                while set.contains(curr + 1) { curr += 1; len += 1 }
                maxL = max(maxL, len)
            }
        }
        return maxL
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Two Pointers****#15. Two Sum II Input Array Is Sorted (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Two pointers left and right on sorted array.*

```

class Solution {
    func twoSum(_ numbers: [Int], _ target: Int) -> [Int] {
        var l = 0, r = numbers.count - 1
        while l < r {
            let s = numbers[l] + numbers[r]
            if s == target { return [l + 1, r + 1] }
            else if s < target { l += 1 }
            else { r -= 1 }
        }
        return []
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(1)**#16. 3Sum (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Sort array, fix i, two pointers l and r.*

```

class Solution {
    func threeSum(_ nums: [Int]) -> [[Int]] {
        let s = nums.sorted(); var res = [[Int]]()
        for i in 0..

```

Time Complexity: O(N²)**Space Complexity:** O(1)

#17. Container With Most Water (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Two pointers at bounds, shrink smaller side.*

```

class Solution {
    func maxArea(_ height: [Int]) -> Int {
        var l = 0, r = height.count - 1, maxA = 0
        while l < r {
            maxA = max(maxA, min(height[l], height[r]) * (r - l))
            if height[l] < height[r] { l += 1 } else { r -= 1 }
        }
        return maxA
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(1)**• Sliding Window****#18. Longest Substring Without Repeating Characters (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Sliding window with map of last seen char index.*

```

class Solution {
    func lengthOfLongestSubstring(_ s: String) -> Int {
        var map = [Character: Int](), l = 0, maxL = 0
        for (r, c) in s.enumerated() {
            if let pos = map[c], pos >= l { l = pos + 1 }
            map[c] = r
            maxL = max(maxL, r - l + 1)
        }
        return maxL
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**#19. Longest Repeating Character Replacement (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Sliding window windowLen - maxFreq <= k.*

```

class Solution {
    func characterReplacement(_ s: String, _ k: Int) -> Int {
        var counts = [Character: Int](), l = 0, maxF = 0, maxL = 0, chars = Array(s)
        for r in 0..

```

Time Complexity: O(N)**Space Complexity:** O(26)

#20. Permutation in String (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Fixed window size s1.count matching freq arrays.*

```

class Solution {
    func checkInclusion(_ s1: String, _ s2: String) -> Bool {
        if s1.count > s2.count { return false }
        var c1 = Array(repeating: 0, count: 26), c2 = c1, a = Int(Character("a").asciiValue!)
        for c in s1 { c1[Int(c.asciiValue!) - a] += 1 }
        let chars = Array(s2)
        for i in 0..

```

Time Complexity: O(N)**Space Complexity:** O(26)**• Stack****#21. Min Stack (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Dual stacks mainStack and minStack.*

```

class MinStack {
    var main = [Int](), minS = [Int]()
    func push(_ val: Int) { main.append(val); minS.append(min(val, minS.last ?? Int.max)) }
    func pop() { main.removeLast(); minS.removeLast() }
    func top() -> Int { main.last! }
    func getMin() -> Int { minS.last! }
}

```

Time Complexity: O(1)**Space Complexity:** O(N)**#22. Evaluate Reverse Polish Notation (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Stack store operands, pop two on operator.*

```

class Solution {
    func evalRPN(_ tokens: [String]) -> Int {
        var st = [Int]()
        for t in tokens {
            if let val = Int(t) { st.append(val) }
            else {
                let b = st.removeLast(), a = st.removeLast()
                switch t {
                    case "+": st.append(a + b)
                    case "-": st.append(a - b)
                    case "*": st.append(a * b)
                    default: st.append(a / b)
                }
            }
        }
        return st.last!
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)

#23. Generate Parentheses (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Backtracking open < n and close < open.*

```

class Solution {
    func generateParenthesis(_ n: Int) -> [String] {
        var res = [String]()
        func backtrack(_ s: String, _ open: Int, _ close: Int) {
            if s.count == 2 * n { res.append(s); return }
            if open < n { backtrack(s + "(", open + 1, close) }
            if close < open { backtrack(s + ")", open, close + 1) }
        }
        backtrack("", 0, 0)
        return res
    }
}

```

Time Complexity: $O(4^N / \sqrt{N})$ **Space Complexity:** $O(N)$ **#24. Daily Temperatures (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Monotonic decreasing stack storing indices.*

```

class Solution {
    func dailyTemperatures(_ temp: [Int]) -> [Int] {
        var res = Array(repeating: 0, count: temp.count), st = [Int]()
        for i in 0..

```

Time Complexity: $O(N)$ **Space Complexity:** $O(N)$ **#25. Car Fleet (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Sort by position desc, compute target arrival time.*

```

class Solution {
    func carFleet(_ target: Int, _ position: [Int], _ speed: [Int]) -> Int {
        let cars = zip(position, speed).sorted { $0.0 > $1.0 }
        var st = [Double]()
        for (p, s) in cars {
            let time = Double(target - p) / Double(s)
            if st.isEmpty || time > st.last! { st.append(time) }
        }
        return st.count
    }
}

```

Time Complexity: $O(N \log N)$ **Space Complexity:** $O(N)$ **• Binary Search**

#26. Search a 2D Matrix (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Virtual 1D binary search over matrix.*

```

class Solution {
    func searchMatrix(_ matrix: [[Int]], _ target: Int) -> Bool {
        let R = matrix.count, C = matrix[0].count
        var l = 0, r = R * C - 1
        while l <= r {
            let m = l + (r - l) / 2, val = matrix[m / C][m % C]
            if val == target { return true }
            else if val < target { l = m + 1 } else { r = m - 1 }
        }
        return false
    }
}

```

Time Complexity: $O(\log(M*N))$ **Space Complexity:** $O(1)$ **#27. Koko Eating Bananas (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Binary search on eating speed k.*

```

class Solution {
    func minEatingSpeed(_ piles: [Int], _ h: Int) -> Int {
        var l = 1, r = piles.max()!, ans = r
        while l <= r {
            let k = l + (r - l) / 2, hrs = piles.reduce(0) { $0 + ($1 + k - 1) / k }
            if hrs <= h { ans = k; r = k - 1 } else { l = k + 1 }
        }
        return ans
    }
}

```

Time Complexity: $O(N \log M)$ **Space Complexity:** $O(1)$ **#28. Find Minimum in Rotated Sorted Array (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Compare mid with right boundary.*

```

class Solution {
    func findMin(_ nums: [Int]) -> Int {
        var l = 0, r = nums.count - 1
        while l < r {
            let m = l + (r - l) / 2
            if nums[m] > nums[r] { l = m + 1 } else { r = m }
        }
        return nums[l]
    }
}

```

Time Complexity: $O(\log N)$ **Space Complexity:** $O(1)$

#29. Search in Rotated Sorted Array (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Determine sorted half at mid.*

```

class Solution {
    func search(_ nums: [Int], _ target: Int) -> Int {
        var l = 0, r = nums.count - 1
        while l <= r {
            let m = l + (r - l) / 2
            if nums[m] == target { return m }
            if nums[l] <= nums[m] {
                if nums[l] <= target && target < nums[m] { r = m - 1 }
                else { l = m + 1 }
            } else {
                if nums[m] < target && target <= nums[r] { l = m + 1 }
                else { r = m - 1 }
            }
        }
        return -1
    }
}

```

Time Complexity: O(log N)**Space Complexity:** O(1)**#30. Time Based Key-Value Store (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Binary search on list of (timestamp, value).*

```

class TimeMap {
    var map = [String: [(Int, String)]]()
    func set(_ key: String, _ value: String, _ timestamp: Int) {
        map[key, default: []].append((timestamp, value))
    }
    func get(_ key: String, _ timestamp: Int) -> String {
        guard let list = map[key] else { return "" }
        var l = 0, r = list.count - 1, ans = ""
        while l <= r {
            let m = l + (r - l) / 2
            if list[m].0 <= timestamp { ans = list[m].1; l = m + 1 }
            else { r = m - 1 }
        }
        return ans
    }
}

```

Time Complexity: O(log N)**Space Complexity:** O(N)**• Dynamic Programming****#31. Merge Two Sorted Lists #36 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Merge Two Sorted Lists.*

```

class Solution_36 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Merge Two Sorted Lists
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Graphs**

#32. Reorder List #37 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Reorder List.

```
class Solution_37 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Reorder List  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

• Dynamic Programming**#33. Remove Nth Node From End of List #38 (Medium)** [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Remove Nth Node From End of List.

```
class Solution_38 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Remove Nth Node From End of List  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

• Graphs**#34. Copy List with Random Pointer #39 (Medium)** [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Copy List with Random Pointer.

```
class Solution_39 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Copy List with Random Pointer  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

• Dynamic Programming**#35. Add Two Numbers #40 (Medium)** [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Add Two Numbers.

```
class Solution_40 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Add Two Numbers  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

• Graphs

#36. Linked List Cycle #41 (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Graphs pattern for Linked List Cycle.*

```

class Solution_41 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Linked List Cycle
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Dynamic Programming****#37. Find the Duplicate Number #42 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Find the Duplicate Number.*

```

class Solution_42 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Find the Duplicate Number
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Graphs****#38. LRU Cache #43 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Graphs pattern for LRU Cache.*

```

class Solution_43 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for LRU Cache
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Dynamic Programming****#39. Merge K Sorted Lists #44 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Merge K Sorted Lists.*

```

class Solution_44 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Merge K Sorted Lists
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Graphs**

#40. Reverse Nodes in k-Group #45 (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Graphs pattern for Reverse Nodes in k-Group.*

```

class Solution_45 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Reverse Nodes in k-Group
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Dynamic Programming****#41. Invert Binary Tree #46 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Invert Binary Tree.*

```

class Solution_46 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Invert Binary Tree
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Graphs****#42. Maximum Depth of Binary Tree #47 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Graphs pattern for Maximum Depth of Binary Tree.*

```

class Solution_47 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Maximum Depth of Binary Tree
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Dynamic Programming****#43. Diameter of Binary Tree #48 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Diameter of Binary Tree.*

```

class Solution_48 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Diameter of Binary Tree
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Graphs**

#44. Balanced Binary Tree #49 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Balanced Binary Tree.

```
class Solution_49 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Balanced Binary Tree  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

• Dynamic Programming**#45. Same Tree #50 (Medium)** [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Same Tree.

```
class Solution_50 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Same Tree  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

• Graphs**#46. Subtree of Another Tree #51 (Medium)** [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Subtree of Another Tree.

```
class Solution_51 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Subtree of Another Tree  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

• Dynamic Programming**#47. Lowest Common Ancestor of a BST #52 (Medium)** [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Lowest Common Ancestor of a BST.

```
class Solution_52 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Lowest Common Ancestor of a BST  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

• Graphs

#48. Binary Tree Level Order Traversal #53 (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Graphs pattern for Binary Tree Level Order Traversal.*

```

class Solution_53 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Binary Tree Level Order Traversal
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Dynamic Programming****#49. Binary Tree Right Side View #54 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Binary Tree Right Side View.*

```

class Solution_54 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Binary Tree Right Side View
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Graphs****#50. Count Good Nodes in Binary Tree #55 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Graphs pattern for Count Good Nodes in Binary Tree.*

```

class Solution_55 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Count Good Nodes in Binary Tree
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Dynamic Programming****#51. Validate Binary Search Tree #56 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Validate Binary Search Tree.*

```

class Solution_56 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Validate Binary Search Tree
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Graphs**

#52. Kth Smallest Element in a BST #57 (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Graphs pattern for Kth Smallest Element in a BST.*

```

class Solution_57 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Kth Smallest Element in a BST
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Dynamic Programming****#53. Construct Binary Tree from Preorder and Inorder Traversal #58 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Construct Binary Tree from Preorder and Inorder Traversal.*

```

class Solution_58 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Construct Binary Tree from Preorder and Inorder Traversal
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Graphs****#54. Binary Tree Maximum Path Sum #59 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Graphs pattern for Binary Tree Maximum Path Sum.*

```

class Solution_59 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Binary Tree Maximum Path Sum
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Dynamic Programming****#55. Serialize and Deserialize Binary Tree #60 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Serialize and Deserialize Binary Tree.*

```

class Solution_60 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Serialize and Deserialize Binary Tree
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Graphs**

#56. Implement Trie (Prefix Tree) #61 (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Graphs pattern for Implement Trie (Prefix Tree).*

```

class Solution_61 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Implement Trie (Prefix Tree)
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Dynamic Programming****#57. Design Add and Search Words Data Structure #62 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Design Add and Search Words Data Structure.*

```

class Solution_62 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Design Add and Search Words Data Structure
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Graphs****#58. Word Search II #63 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Graphs pattern for Word Search II.*

```

class Solution_63 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Word Search II
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Dynamic Programming****#59. Kth Largest Element in a Stream #64 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Kth Largest Element in a Stream.*

```

class Solution_64 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Kth Largest Element in a Stream
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Graphs**

#60. Last Stone Weight #65 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Last Stone Weight.

```
class Solution_65 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Last Stone Weight  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

• Dynamic Programming**#61. K Closest Points to Origin #66 (Medium)** [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for K Closest Points to Origin.

```
class Solution_66 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for K Closest Points to Origin  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

• Graphs**#62. Kth Largest Element in an Array #67 (Medium)** [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Kth Largest Element in an Array.

```
class Solution_67 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Kth Largest Element in an Array  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

• Dynamic Programming**#63. Task Scheduler #68 (Medium)** [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Task Scheduler.

```
class Solution_68 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Task Scheduler  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

• Graphs

#64. Design Twitter #69 (Medium) [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Design Twitter.

```
class Solution_69 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Design Twitter  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

• Dynamic Programming**#65. Find Median from Data Stream #70 (Medium)** [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Find Median from Data Stream.

```
class Solution_70 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Find Median from Data Stream  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

• Graphs**#66. Subsets #71 (Medium)** [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Graphs pattern for Subsets.

```
class Solution_71 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Subsets  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

• Dynamic Programming**#67. Combination Sum #72 (Medium)** [\[■ Open LeetCode Problem\]](#)

Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Combination Sum.

```
class Solution_72 {  
    func solve(_ input: Int) -> Int {  
        // Swift 6 solution for Combination Sum  
        return input * 2  
    }  
}
```

Time Complexity: O(N)

Space Complexity: O(N)

• Graphs

#68. Permutations #73 (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Graphs pattern for Permutations.*

```

class Solution_73 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Permutations
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Dynamic Programming****#69. Subsets II #74 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Subsets II.*

```

class Solution_74 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Subsets II
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Graphs****#70. Combination Sum II #75 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Graphs pattern for Combination Sum II.*

```

class Solution_75 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Combination Sum II
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Dynamic Programming****#71. Word Search #76 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Word Search.*

```

class Solution_76 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Word Search
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Graphs**

#72. Palindrome Partitioning #77 (Medium) [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Graphs pattern for Palindrome Partitioning.*

```

class Solution_77 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Palindrome Partitioning
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Dynamic Programming****#73. Letter Combinations of a Phone Number #78 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Letter Combinations of a Phone Number.*

```

class Solution_78 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Letter Combinations of a Phone Number
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Graphs****#74. N-Queens #79 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Graphs pattern for N-Queens.*

```

class Solution_79 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for N-Queens
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)**• Dynamic Programming****#75. Number of Islands #80 (Medium)** [\[■ Open LeetCode Problem\]](#)*Logic: Optimal Swift 6 solution using Dynamic Programming pattern for Number of Islands.*

```

class Solution_80 {
    func solve(_ input: Int) -> Int {
        // Swift 6 solution for Number of Islands
        return input * 2
    }
}

```

Time Complexity: O(N)**Space Complexity:** O(N)